PPT-LAT-Systems

Shannon-Prime Lattice: Systems Architecture

A practical paper for the shannon-prime-lattice project. Restates engine backends, inline compression, model-family coverage, gated sieve/ARM/DHT features, and a formal blockchain design under one umbrella. Mathematical underpinnings are referenced but not derived; this paper is about the system you build and ship.

Abstract

Shannon-Prime Lattice is an inference and coordination substrate. It runs large transformer language models on hardware ranging from commodity desktops to Snapdragon-class phones, compresses the things that make inference expensive (weights, KV cache, gradients), and optionally federates participating nodes into a global compute fabric whose contributions are accounted for on-chain. The mathematical core has been argued elsewhere: the same lattice object underlies token positions, attention scores, weight quantization, KV-cache compression, sieve dominance, and sharded inference. The job of this document is the *systems* story. It describes four engine backends (CPU, CUDA, Vulkan, Hexagon), the inline compression machinery (Q8 weights, Q4 mixed-precision, VHT2 KV cache), model-family support (Llama, Qwen3-series, Gemma 2.5/3/4, DeepSeek V4), the *gated* Lattice features (sieve, ARM, CRT sharding, DHT, token economy), and finally the blockchain design that gives federated participation its accounting layer. Throughout, one invariant is binding: every Lattice-only feature must be off-by-default and the unfeatured baseline must be bit-identical to a plain inference path. The blockchain section is presented as scaffolding, not specification — it is the part of the design we expect to revise most often as constants meet adversarial reality.

Overview: six layers of one math object

A working mental picture of the system: there is one mathematical object — a discrete lattice over a cyclotomic ring, with a CRT decomposition into two coprime Proth primes and an NTT diagonalization that turns convolution into pointwise product — and that single object reappears six times as one walks up from silicon to network.

- At the silicon layer, it is the choice of vector width and integer precision. The NTT primes are sized so that pointwise multiplication fits inside whatever the underlying ALU offers: 32-bit on CPU, 30-bit halves on Hexagon HVX.
- At the kernel layer, it is the matmul: weight tiles in packed integer storage, activations in fp32 or bf16, an inline dequant step that consumes the Frobenius row scale before accumulating.
- At the cache layer, it is the KV representation: K and V are projected onto lattice anchors, stored as small integer blobs, and decoded only when read.
- At the model layer, it is the attention computation: queries and keys live in the same ring, so dot products become polynomial multiplications and we can use cached transforms.
- At the sieve layer, it is the dominance partial order: cached cells form a poset under componentwise lattice comparison, and the engine keeps the dominance-incomparable frontier.
- At the network layer, it is the routing key space: peers hash into slabs indexed by prime factorization, and shards of inference flow over a DHT whose addresses are residues of the same primes that ground the NTT.

The point is not that each layer is metaphysically the same — it is that we exploit the structure of the lattice consistently, so a careful implementation at one layer composes correctly with the layer above and below. The system gets simpler as you understand it more, not more complex.

Section 1: Engine backends

The engine is portable. The same forward pass runs on four backends, dispatched by a thin runtime selector. Each backend implements the same internal kernel surface: a packed-weight matmul, a fused softmax-attention, an FFN with SwiGLU, a positional-encoding op, and the NTT primitive used by attention and (optionally) by sieve operations. Loaders and tokenizers are shared.

1.1 CPU backend

The CPU backend is the reference implementation and the production target for commodity desktops and laptops. It builds on Windows with MSVC (VS2019 Build Tools, MSVC v142 toolset) and on Linux with gcc 11+ or clang 14+. There are two vectorized paths: AVX2 (8-wide single precision, available on essentially every x86-64 chip from the last decade) and AVX-512 (16-wide single precision, plus VNNI for INT8 dot products on capable Intel and AMD Zen 4+ parts). The scalar path remains as a correctness baseline; it is what unit tests check against and what the CRT NTT branches use as an alternate residue to detect arithmetic forgery on the chain.

For matmul, the CPU backend operates row-major-by-token on the activation side and column-major-by-token on the cache side. This asymmetry came out of profiling and is load-bearing: it lets attention prefill batch tokens contiguously while keeping the on-write transposition of K and V cheap. AVX-512 matmul tiles 16 output channels at a time, prefetching the next packed weight tile while accumulating into a register file held in zmm registers. The AVX2 variant tiles 8 channels at a time and uses a software-pipelined prefetch lag of two tiles. Both paths handle Q8 dequant inline: a packed byte arrives, is sign-extended to int16, multiplied by the row's Frobenius scale (a single fp32 broadcast), and added to the running fp32 accumulator. There is no decode arena and no intermediate buffer; the bytes are read directly off the packed weight storage.

For attention, the CPU backend has two modes. In dense mode it computes the full QK^T softmax V chain at fp32. In poly-ring mode (default at large head dimensions) it transforms Q and K into the NTT domain, computes pointwise products there, and inverse-transforms only the output. The NTT itself uses Barrett reduction with precomputed mu constants; we measured 4x kernel speedup from this versus a naive Montgomery layout. The K cache holds its NTT-domain transform persistently across forward steps, so generation never re-transforms a key that has already been seen.

For FFN, SwiGLU is fused: the gate projection, up projection, silu, and elementwise multiply happen in a single pass over the hidden state, so the activation never has to round-trip through DRAM between sub-ops. This is meaningfully faster on memory-bandwidth-bound CPUs.

Threading is via standard pthreads or std::thread; the engine's job pool is sized to one worker per physical core minus one (the conductor thread). NUMA-aware placement is supported but off by default; on dual-socket servers, enabling it gives roughly 1.4x on prefill but introduces locality bugs we are still characterizing.

A few additional CPU-backend details that matter to operators. First, the build is statically linked by default — we don't depend on any shared library at runtime beyond the C runtime and a small portable threading shim. This avoids the usual library-versioning hell on Linux

distributions and on stock Windows installs without redistributables. Second, the binary is single-file: the model is a separate file (the GGUF), the engine is one executable, and the configuration is environment variables and command-line flags. There is no install step. Third, the engine respects two affinity hints — `SP_CPU_CORES=N` to cap the worker pool at N workers, and `SP_CPU_AFFINITY=mask` to pin workers to a specific core mask. These are operator escape hatches; the auto-detected defaults are correct for almost every machine.

One subtle behavior worth calling out: the CPU backend will silently fall back from AVX-512 to AVX2 if it detects an AVX-512 frequency-throttling event on Intel Skylake-X and Ice Lake parts. The detection is heuristic — we measure the wall-clock cost of a fixed-size matmul during warmup and compare AVX2 versus AVX-512; if AVX-512 is slower by more than 10% we stay on AVX2 for the rest of the session. This came out of bench results where AVX-512 should have been faster on paper but was slower in practice because of the throttling. The behavior can be disabled with `SP_CPU_FORCE_AVX512=1` for the benchmarker who knows what they want.

1.2 CUDA backend

The CUDA backend targets RTX 3000 and 4000 series consumer cards. It builds with NVCC + MSVC + CUDA 12.4, currently against compute capability 8.6 (Ampere) and 8.9 (Ada Lovelace). The build system requires Visual Studio 2019 Build Tools and the `--use-local-env` flag through CMake; this is brittle but documented in the repo, and shaves roughly five minutes off the configure step compared to the recommended path.

Kernels are implemented in CUDA C++. Matmul uses warp-cooperative tiles: each warp owns a 16x16 output block, with shared-memory tiles for the weight side and register tiles for the activation side. We do not currently use Tensor Cores; the Q8 dequant path is not yet expressed in a way that maps to mma.sync, and the gain from converting it would be modest until we also push activations into bf16, which is a separate workstream. NTT is implemented as a Cooley-Tukey radix-2 butterfly with the twiddle factors held in constant memory; for the configured N=256, the entire butterfly fits in a single CTA and the NTT cost is dominated by the memcpy in/out, which we plan to eliminate by fusing NTT into the K cache writeback.

Attention has two implementations: a fused-softmax dense kernel and a poly-ring kernel. The fused-softmax kernel is competitive with FlashAttention v2 at sequence lengths up to about 4K on Ampere; beyond that, FlashAttention's online softmax becomes important and we lose ground. We have a port of online softmax in flight but have not validated it for arithmetic parity with the CPU reference yet, which is required before it can be used by chain validators.

Falls back to CPU for unsupported ops, which currently includes the sieve dominance check (it's a CPU op anyway; lattice comparisons branch too much to be GPU-friendly) and the freethedsp/Hexagon-specific code paths (obviously). The fallback is automatic and silent; a debug flag (`SP_DEBUG_FALLBACK=1`) prints when it engages.

Multi-GPU is supported via a tensor-parallel layout across NVLink-connected cards. The simplest form, two-way TP, splits each attention head's QKV across two GPUs and the FFN intermediate dimension across two GPUs; the all-reduce happens at the residual-stream junction. We have validated this on a pair of RTX 4090s for Llama 3.1 8B and Qwen3 7B and seen close to linear scaling on prefill, with generation showing the expected NCCL latency tax. Four-way and eight-way TP are coded but not validated on consumer hardware (we lack the boxes); they should work on data-center cards. Pipeline parallelism is not implemented and not planned in the immediate term; we prefer the simplicity of pure TP at the scales we target.

GPU memory budgeting matters because consumer cards are tight. For Llama 3.1 8B at Q8 with 32K context and KV compression on, we need roughly 9 GB on a single RTX 3090/4090 (24 GB), which leaves plenty of headroom. For Llama 3.1 70B at Q8 we cannot fit a single card; two-way TP across two 24 GB cards is the minimum, and four-way is comfortable. For the 671B

DeepSeek V4, no consumer card configuration fits the model resident; expert streaming from NVMe is mandatory, and a fast PCIe Gen 4 NVMe is the binding constraint on per-token latency.

1.3 Vulkan backend

The Vulkan backend exists to reach AMD and Intel GPUs and to give us a portable GPU baseline that does not require CUDA. It builds against the Vulkan SDK 1.3.x and uses glslc for shader compilation. Compute shaders implement matmul, NTT butterflies, attention, and FFN; the same SPIR-V binaries run on Windows, Linux, and (in principle) Android, though the Android path is not yet wired into the production loader.

Vulkan is a lower priority than CUDA today because we do not have validated production results on AMD parts at the Llama-3 scale, and because the shader-compilation step adds a measurable cold-start cost (several seconds on first run; cached after that). However it is the only viable cross-platform GPU path, and it is what will eventually run on Intel Arc, Apple silicon (via MoltenVK), and Adreno mobile GPUs. The engine treats Vulkan symmetrically with CUDA at the dispatch level: same kernel surface, same fallback semantics.

One specific Vulkan caveat: subgroup operations are not uniformly supported across vendors. We currently use only the subgroup ballot and shuffle intrinsics that are guaranteed by the Vulkan 1.2 core spec, which forces some matmul reductions onto shared-memory paths that would be cheaper as warp shuffles in CUDA. The gap closes once we are willing to require Vulkan 1.3 device extensions for AMD RDNA3 and Intel Xe2; we have not made that call yet.

1.4 Hexagon backend

The Hexagon backend is the phone path. It targets the Snapdragon 8 Gen 1's V69 Hexagon Tensor Accelerator, accessed via FastRPC from the Android ARM cores. The build environment is the Hexagon SDK 5.x on a Windows host, with the specific patch documented in the build recipe (the SDK's `hexagon_fun.cmake` points at `bin/qaic`, but on Windows the binary is `WinNT/qaic.exe`, and the Git `sh.exe` has to be prepended to PATH so the shell wrappers resolve correctly).

The backend has three logical layers, used in different combinations depending on what the device exposes:

- **Halide-compiled HVX kernels.** These are the hand-rolled vector kernels: matmul, Q8 dequant, NTT, attention. HVX is 128-byte vectors operating on fixed-point INT8/INT16 lanes; the matmul kernel produces 32 output channels per HVX vector load. Halide is the source language; it is the same code that has been validated on the S22U at 81 us minimum dispatch time, which matches the AI Hub reference run for our test model.
- **QNN HTP backend.** QNN (Qualcomm Neural Network SDK) lets us schedule operations on the V69 HTP using either AOT-compiled .bin graphs or runtime-built graphs constructed via `QnnGraph_addNode`. The runtime path is the production target because it lets us reconfigure the matmul shape per-layer without a recompile. The dispatch ceiling we measured is roughly 577 calls/sec over FastRPC; this is the hard upper bound on per-token throughput at fine granularity. Our 4-split-per-token layout sits comfortably under that ceiling at 38.7 t/s.
- **freethedsp shim.** Production-locked Android devices restrict signed-PD attachment to apps with the right certificates. The freethedsp shim, adapted from public work in the community, opens the DSP via an unsigned PD by LD_PRELOAD'ing into the FastRPC client. It is opt-in via `SP_FREETHEDSP=1`; the default path uses signed PD where available. The shim is the only way to bench on a stock retail device without unlocking the bootloader.

Mode C of the Hexagon backend (QNN HTP for matmul, HVX for everything else) is the current production target. Mode D (the ISP-as-spectral-reconstructor pipeline, where the Spectra 680

image-signal-processor performs skeleton+residual band fusion at 18-bit fixed point in parallel with HTP matmul) is research; the pieces have been validated individually but not strung together.

FastRPC scratch buffer allocation has one footgun we hit repeatedly: the rpcmem registration size must exactly equal the IDL length parameter. Over-allocation silently fails with `AEE_EUNSUPPORTED`, which looks like a totally unrelated DSP error. This is documented in the repo and worth repeating here because every new contributor hits it.

A second Hexagon-specific note: thermal management. The V69 HTP can run at full power for short bursts but throttles after sustained load — typically after 30-60 seconds of continuous inference on a phone with passive cooling. The throttle drops throughput by roughly 40%. Our default scheduler accommodates this by inserting micro-pauses (1-2 ms) between layers when the host thermal sensor reports the SoC entering a warning zone. The pauses are cheap on the per-token critical path but visibly reduce throughput; an operator who needs sustained peak performance on a phone needs an external cooling solution, which is mostly a research-rig configuration. Production usage on a normal phone is bursty (a few seconds of inference followed by user reading time) and rarely hits the throttle.

Memory on the phone is the other binding constraint. On the S22U test rig (12 GB total system RAM, of which roughly 8 GB is available to user processes), we can fit a Qwen3-3B at Q4 with the full Lattice cache enabled. The 7B class is uncomfortable; 8B requires aggressive Q4 across all layers and trims context to 8K. Larger phone SoCs (24 GB on the upcoming flagship class) loosen this enough that 7B-class models become comfortable; we benched a preview device and saw the expected linear scaling. The Hexagon backend is therefore well-positioned for the model-size class that will dominate consumer use for the next 18 months, which is roughly 1B-7B.

1.5 Common loader and dispatcher

GGUF is the on-disk model format. The loader is shared across all four backends; it memory-maps the file, validates the header, walks the tensor table, and produces a uniform tensor descriptor that the backend-specific dispatcher consumes. The descriptor carries dtype, shape, role (e.g., “attn.wq.layer_5”), and any quantization metadata (Frobenius scale for Q8, calibration constants for Q4). Loader extensions are how new model architectures are added: a new arch declares its tensor inventory and the per-tensor roles, and the dispatcher routes ops accordingly.

The dispatcher itself is a small function-pointer table per backend. At engine init, the selected backend populates the table; the forward pass then calls through the table without any per-op branching. This is what makes backend-swap cheap: there is no scattered `if (backend == CUDA)` logic anywhere in the model code.

Backend	Build env	Targets	Matmul	NTT	Attention	KV compress
CPU AVX2	MSVC / gcc	x86-64 universal	yes	scalar	dense + poly	yes
CPU AVX-512	MSVC / gcc	Skylake-X, Zen 4+	yes	Barrett SIMD	dense + poly	yes
CUDA	NVCC + MSVC, CUDA 12.4	Ampere, Ada	yes	radix-2 butterfly	fused + poly	partial
Vulkan	Vulkan SDK 1.3 + glslc	AMD, Intel, mobile	yes	butterfly shader	fused	partial

Backend	Build env	Targets	Matmul	NTT	Attention	KV compress
Hexagon HVX	Hexagon SDK 5.x	V69 HTP (SD8G1)	yes	INT16 fixed	poly	yes

1.6 Three-Step Relative Attention Cache

Standard Rotary Positional Embeddings (RoPE) compute relative phase shifts using continuous floating-point trigonometry, leading to immense cache sizes at long contexts. The engine exploits the Three-Gap Theorem (PPT-LAT-Theory T7) to collapse this. Because our frequencies are derived from optimal irrational rotations, the phase shifts $\Delta \cdot \theta_d$ for any relative offset Δ, when sorted by frequency, exhibit at most three distinct discrete increments.

By re-ordering the Q/K projection dimensions to align contiguous frequency phases with the VHT2 + Möbius squarefree anchors, relative attention reduces from a continuous calculation to a discrete combinatorial jump. The dimension-ordering decision (raised as a design question in earlier drafts) is closed: the sort key for the three-step structure is the *same* sort key VHT2 + Möbius already uses for the squarefree anchors, so no second sort is required — the continuous phase rotation natively aligns with the discrete structural lattice.

- **Memory reduction.** For $D = 128$ and ctx $= 4096$, a naive relative-attention cache requires $4096 \times 128 = 524{,}288$ entries. The Three-Step architecture requires $4096 \times 3 = 12{,}288$ entries — a ~43× memory reduction.
- **Compute reduction.** Continuous trigonometric functions are stripped from the inference loop, replaced by composing one of three precomputed rotation matrices per relative offset.
- **Backend impact.** All four backends benefit; the win is largest on CPU (where trig calls dominate) and Hexagon (where transcendentals are expensive).
- **Implementation gate.** Default OFF until per-backend parity is established. Engaged via `SP_ROPE_THREE_STEP=1`. The regression invariant holds: with the gate off, the engine produces bit-identical output to the standard RoPE path.

Section 2: Inline weight and cache compression

Compression is foundational, not an option. Everything else assumes weights are packed and the KV cache is encoded; the unpacked path exists for validation and is not optimized.

2.1 Q8 weight storage with Frobenius scale

Each weight tensor is stored as a stream of packed bytes plus a per-row scale array. The packing is straightforward: each fp32 row is normalized by its largest absolute value, scaled into the [-127, 127] range, rounded with ceiling-shift to avoid round-half-up bias, and stored as signed int8. The scale (a single fp32 per row) lives alongside the packed bytes.

Decompression is inlined into matmul. For each output element, the kernel reads the corresponding packed weight byte, sign-extends to int16, multiplies by the row's scale (broadcast to fp32), and accumulates into the output. On AVX-512, this is one zmm load of 64 packed bytes, one sign-extend to two zmm registers of int16, two multiplies against a broadcast fp32 scale, and an accumulate. On HVX, it is one vector load of 128 packed bytes, two vector multiplies against a precomputed INT16 scale, and an accumulate. There is no intermediate decode arena; the packed bytes flow directly from DRAM into ALU lanes.

The memory saving is 4x versus fp16 and 8x versus fp32. On the Gemma3-1B test rig, this brought a 10.4 GB weight footprint down to 1.3 GB, which matters because it leaves headroom for KV cache, scratch buffers, and the lattice sieve on devices where DRAM is the binding constraint. The accuracy cost is small: production logs show Q8 perplexity within 0.6% of the fp16 baseline on a calibrated chunk-4 evaluation. The cost is small enough that we ship Q8 as the default and treat fp16 as a debugging mode.

There is a background prefetcher path for Q8 weights: a dedicated thread maintains a two-slot double buffer ahead of the forward pass, with the forward path acquiring a layer at a time via condvar handoff. The architecture is in production but the speedup is not yet visible at Gemma3-1B scale because of memory-bandwidth contention between prefetcher writes and matmul reads. Larger models, where the per-layer compute time grows faster than the per-layer fetch time, are where this pays off, and that work is queued.

2.2 Q4 mixed-precision path

Q4 is the same architecture as Q8 but with 4-bit packed nibbles, two per byte. The dequant arithmetic is identical: nibble, sign-extend, multiply by row scale, accumulate. The packing density is 16x over fp32 and 8x over fp16, which makes it possible to fit large dense models on phone-class devices.

The catch is accuracy. A first-pass Q4 implementation with a single per-tensor scale produced catastrophic signal collapse on the same Gemma3-1B test rig (perplexity blew up by nine orders of magnitude). The cause was the choice of scale granularity: Q4 has so few discrete levels that a tensor-wide scale clips meaningful variation. The current path is per-row scales (one fp32 per row), the same as Q8, with a calibration pass that selects a clipping percentile rather than the row max. Under this scheme, early bench shows perplexity within roughly 3% of fp16, but the validation work to make this production-ready is not done.

Q4 is therefore opt-in via `--frobenius-q4` and is not the default. It will become default when (a) we have a calibration sweep across all supported model families showing PPL within 1% of fp16 on calibration data and (b) we have a backstop calibration-failure detector that refuses to load a Q4 model that didn't calibrate well.

Mixed precision means the heaviest layers (attention output proj, FFN gate/up/down) stay Q8 while the dominant memory cost (the wq/wk/wv stack and the embedding) goes Q4. The mix is selected per arch by a config file; for Llama-3.2 we mix to roughly 5-bit average; for Qwen3-MoE the expert weights are aggressively Q4 while the router stays Q8 because router miscalibration tanks expert routing quality.

2.3 Inline KV cache compression (VHT2 + Spinor)

KV cache is the second large memory cost after weights, and at long context it dominates. The Lattice cache uses a three-stage encoder:

1. **VHT2 projection.** Each new K vector is projected against a fixed set of lattice anchors. The projection is computed in fp32 and quantized to a small integer per anchor.
2. **Mobius mixing.** A short mixing step rebalances anchor mass across siblings on the lattice tree, reducing the worst-case quantization error.
3. **Spinor packing.** The final representation is a 63-byte Spinor block per cache cell: a small header (norm, exponent), the anchor coefficients (packed integers), and a checksum byte. 63 bytes per cell against the roughly 8 KB an uncompressed fp16 cell would take at d=4096 is the roughly 130x cache compression we cite in benchmarks.

Decoding happens on read, inside the attention kernel. The dequant logic is essentially the same as Q8 weights — read packed integers, multiply by anchor weights, accumulate — except

the anchor table is small and shared across all cache cells, so it fits in L1 on the CPU and in shared memory on the GPU.

The accuracy story for KV compression is harder than for weights. Cache state accumulates errors across timesteps, so even a modest per-cell quantization error compounds in long-context settings. Our mitigation is a periodic *refresh*: every K tokens, the cache cells for tokens older than K are recompressed with a slightly larger Spinor block (96 bytes instead of 63), trading some compression for stability. The refresh interval is tunable; the default of K=512 keeps perplexity drift under 1% out to 32K context on the models we've tested.

The KV compression path is engaged by default at long context (n_ctx > 2048) and bypassed for short prompts where the savings aren't worth the encode overhead.

A further question we get often: why not just use one of the existing quantized KV approaches (KIVI, KVQuant, the GPTQ-K family)? The honest answer is that we evaluated them and found that they trade compression ratio for either accuracy or compute cost in ways that don't suit our workload. KIVI is comparable to our scheme at 4-bit and is simpler to integrate; but it doesn't compose with the lattice sieve, because its cell representation is not a dominance-comparable structure. KVQuant's per-channel scales are competitive but require a calibration pass we don't always have time to run for a freshly-loaded model. Our VHT2+Spinor encoder is the design that gives us the dominance comparability the sieve needs, with a compression ratio in the same neighborhood as the published alternatives. The cost is a more complex encoder; we believe that cost is justified by the downstream gating opportunities the sieve enables. If we are wrong about the value of the sieve, the KV encoder is the easiest part to swap; the abstraction boundary between the encoder and the rest of the system is clean.

Fibonacci sub-sampling for KV eviction. When the context window exceeds memory bounds, the engine employs Fibonacci sub-sampling for KV eviction instead of FIFO or LRU. Standard eviction schemes degrade long-range attention resolution because they bias toward retaining either the oldest or the most-recently-used tokens, leaving gaps in the middle of the context where retrieval queries often need to land. By Theorem T7 (PPT-LAT-Theory), retaining tokens at positions $\lfloor k\varphi \cdot N \rfloor \pmod N$ where $\varphi = (\sqrt{5}-1)/2$ guarantees the retained cache maximally and uniformly covers the temporal context regardless of N, with bounded discrepancy on the temporal axis to mirror the frequency-axis equidistribution exploited by E9.1. Engaged via SP_KV_FIB=1; default OFF until the long-context PPL-drift sweep validates parity with the existing refresh-window scheme.

2.4 Per-hardware code paths

Different vector widths and dequant intrinsics matter enough that we keep separate code paths for each:

AVX2. 8-wide fp32 matmul, integer Q8 dequant via sign-extend and broadcast scale. Compatible across the entire Haswell-and-later x86 world. The fallback when AVX-512 isn't available.

AVX-512. 16-wide fp32 matmul. Uses VNNI (`vpdpbusd`) for INT8 dot products when available, falling back to a manual `vpmaddubsw` + `vpaddw` chain when VNNI isn't. The BF16 fast path is also AVX-512-only; we use it for KV cache value side, where the precision loss versus fp16 doesn't materially affect attention quality.

HVX (Hexagon). 128-byte vectors, fixed-point INT8/INT16 lanes. The HVX matmul kernel operates on 32 output channels per HVX vector and accumulates into a wide register pair. Q8 dequant on HVX uses the `Vh_vmpyio_VhRh` pattern for the int16 multiply against a broadcast scale.

DSP scalar. Fallback for ops that don't have a vector form on the DSP (rare in production; most ops vectorize). The scalar path is also what we use for low-volume control logic like the per-layer norm.

ARM NEON. Used on the phone's ARM cores for ops that don't go to the DSP — tokenization, sampling, the host-side glue around FastRPC calls. NEON Q8 dequant uses the standard `vmovl_s8` + `vmulq_f32` pattern; we don't currently use Apple-specific instructions, which means the same code runs on Snapdragon and on Apple silicon via Rosetta translation when needed.

Section 3: Model-family support

Each model family is a forward-pass dispatcher plus a loader extension plus a quantization-sweep validation. The engine's job is to provide the kernels; the family-specific job is to wire them up correctly.

3.1 Llama

Llama is the reference architecture. The engine currently supports Llama 3.1 and 3.2 in production. The forward pass is the standard transformer: RMSNorm, RoPE, grouped-query attention with 8:1 group ratio in the larger sizes, SwiGLU FFN. RoPE is implemented with precomputed sin/cos tables; the GQA cache layout uses a shared K/V slab per group rather than duplicating, which gives the expected 8x cache savings at the 8B size.

Llama 3.2 introduced a tied embedding for the 1B/3B sizes; the loader detects this and reuses the embedding tensor for both input embed and LM head. This is small but easy to get wrong: the LM head still needs the final RMSNorm before projection, even when the projection matrix is the embedding.

The Q8 sweep on Llama 3.1 8B shows PPL 7.18 vs an fp16 baseline of 7.13 (+0.7%), within the 1% bar. The Q4 sweep is not done yet for 3.1; for 3.2 1B we have Q4 at PPL 13.42 vs fp16 13.05 (+2.8%), which is over our bar but not catastrophic. Reasoning: 1B is small enough that Q4 quantization noise is a significant fraction of the model's total expressive capacity. The same arch at 8B is expected to handle Q4 better, but the sweep is queued.

3.2 Qwen3 series

The Qwen3 family is the SOTA target family. Production support exists for:

- **Qwen3** (0.5B, 1.5B, 3B dense). Standard transformer; the main quirk is the tokenizer (BBPE with a specific Chinese-heavy pretraining vocabulary that affects byte-fallback behavior). Q8 PPL within 0.5% of fp16; Q4 within 2% at 3B size.
- **Qwen3.5** (3B, 7B dense). Identical to 3 architecturally; the loader treats them as one family with version-tagged norm constants. The pretraining diet shifted toward more code, which means Qwen3.5-Coder is a particularly strong target for spec-decode (small draft model + bigger verifier).
- **Qwen3.6 MoE**. This is where the family diverges. MoE routing adds an expert-selection gate that runs once per token per layer; the engine implements it as a top-K softmax against the router weights, with K=2 in production. Each expert is a separate FFN slab in the GGUF; the loader walks all of them and packs them as a contiguous Q4 expert pool with a shared scale array. The dispatcher's FFN path becomes a switch: route to expert(s) by the gate's top-K, run the FFN on those experts, combine outputs weighted by gate probabilities. Production Q8 PPL on Qwen3.6-30B-A3B sits within 1.2% of fp16 — a slightly larger gap than dense models because router miscalibration under quantization is a meaningful source of error.

- **Qwen3.7** (announced; not yet supported). The architectural disclosures so far suggest a hybrid SSM+MoE block (similar in spirit to Qwen3-Next), gated attention, and an mRoPE mode 8. Loader extension scope is small but not yet done.

The Qwen series is also where we have the best spec-decode results: a 0.5B draft model running ahead of a 3B verifier hits 2.16x spec-decode speedup, and stacking with the Hexagon fused KQ path on the verifier brings that to 2.23x. Spec-decode interacts with KV compression in a subtle way: the draft and the verifier must agree bit-exactly on the verified prefix, which means their KV caches must use the same encoder configuration. We enforce this by requiring both models to be loaded with the same `SP_LATTICE_SIEVE` value.

3.3 Gemma 2.5/3/4

Gemma's signature is its attention pattern: a sliding-window attention interleaved with full attention, in a 5:1 local-to-global ratio in Gemma 3. This means most layers see only the last W=4096 tokens, while every sixth layer attends to the full context. The dispatcher's attention path consults a per-layer flag that selects between the local and global kernel. Both use the same poly-ring attention math; the only difference is the SWA mask bounds.

Gemma 2.5 adds post-attention and post-FFN norms — RMSNorm not just on the residual input but on the residual output. The loader sets up four norm slots per block instead of two; the dispatcher's block path runs them at the appropriate points. The cost is small but the order matters; a wrong-order norm produces a model that is coherent-sounding but measurably worse on PPL benchmarks, which makes the bug easy to miss without a proper sweep.

Gemma 3 added the local/global pattern. Gemma 4 (in preview) reportedly extends the SWA window and adds a routing-style gate to the FFN, but we have not yet seen a checkpoint we can load.

Q8 PPL on Gemma 3 4B sits within 0.4% of fp16. The Gemma family benefits unusually well from KV compression because the SWA window already bounds the working cache size; the Lattice cache then compresses *within* the window, so per-token cache cost drops by another order of magnitude.

3.4 DeepSeek V4

DeepSeek V4 is the deep end. 671B parameters MoE with roughly 37B active per token. FP8 training native, which means the published checkpoints are FP8 from the start; converting to fp16 for our standard pipeline doubles the disk footprint. Multi-token prediction (MTP) lets each forward pass produce K candidate next tokens that the model itself can verify; this is a form of self-spec-decode that interacts cleanly with our Lattice cache.

The DeepSeek MoE routing is more complex than Qwen's. It uses a learned load-balancing loss during training that biases the gate toward a balanced expert assignment; at inference, we still take top-2, but the gate has more dynamic range, which means the top-2 weights are closer to (0.5, 0.5) than to (0.8, 0.2). This matters for quantization: the secondary expert contributes meaningfully to the output, so Q4 on experts is more sensitive than on Qwen3.6.

The engine's DeepSeek V4 support is partial. We can load and run the model on the CUDA backend at full size, with experts streamed from disk because they don't fit in any commodity GPU's VRAM. Streamed-expert inference is slow — a single forward pass is dominated by SSD bandwidth, not compute — but it works, and it is the prerequisite for the CRT-sharded multi-node inference path described later. The Hexagon backend cannot host DeepSeek V4 at all; the model is simply too large for any phone-class device. The Vulkan backend is gated on the streamed-expert infrastructure being ported to its dispatcher, which is on the roadmap.

DeepSeek V4 is also our test case for the most aggressive compression configurations: Q4 experts, Q8 attention, BF16 KV cache. We have an internal eval running showing PPL within 2% of FP8 baseline on a subset of HumanEval and MMLU; the full sweep is pending. This is the model where the next year of optimization will pay the most attention.

Family	Sizes	Backends	Q8 PPL gap	Q4 PPL gap	Notes
Llama 3.1	8B, 70B	CPU, CUDA	+0.7%	n/a yet	reference arch
Llama 3.2	1B, 3B	all four	+0.5%	+2.8% (1B)	tied embed
Qwen3	0.5B-3B	all four	+0.5%	+2%	best draft for spec
Qwen3.5	3B, 7B	CPU, CUDA, Vulkan	+0.6%	+2.1%	code-focused
Qwen3.6 MoE	30B-A3B	CPU, CUDA	+1.2%	sensitive	top-2 routing
Qwen3.7	tba	unsupported	-	-	hybrid SSM+MoE
Gemma 2.5	2B, 9B	CPU, CUDA	+0.4%	+1.8%	post-attn/FFN norms
Gemma 3	1B, 4B, 12B, 27B	all four	+0.4%	+1.5%	SWA 5:1
Gemma 4	tba	unsupported	-	-	preview
DeepSeek V4	671B-A37B	CUDA (streamed)	+2% (partial)	sensitive	FP8 native, MTP

Section 4: Sieve / Lattice features — gated

This is the part of the system where the engine becomes Lattice. Up to this point, everything described is a fast, compressed, portable inference engine. The features in this section turn it into a node in a federated fabric. They are all gated, off by default, and bypassed entirely in the baseline path. The *regression invariant* is binding: if every gate is off, the engine produces output that is bit-identical to a plain inference path. We test this every release with a parity sweep across the four backends.

4.1 KSTE-encoded KV cache (`SP_LATTICE_SIEVE`)

KSTE (Key-Side Tree Encoding) takes a KV cache cell and emits a small structured tree: the root holds a coarse fingerprint of the cell, internal nodes hold residual error against the parent fingerprint, and leaves hold the Spinor block. The tree representation has two purposes. First, it makes dominance comparison cheap: two cells can be ordered by their tree fingerprints without decoding the leaves. Second, it makes deduplication via the Friedman sieve straightforward: the sieve keeps only the dominance-incomparable trees, dropping cells whose trees are strictly dominated by another cell already in the cache.

The Friedman sieve runs on cache write. When a new cell arrives, the sieve walks the existing frontier (typically a few hundred entries deep) and checks whether the new cell dominates or is dominated by any frontier entry. Dominated cells are dropped; dominating cells push out the existing entries they dominate. The complexity is O(frontier size) per insert, which is acceptable up to frontiers of a few thousand cells, beyond which we keep a coarser top-level index by slab.

With the gate off, KV writes go straight into the standard VHT2+Spinor encoder; with the gate on, they additionally pass through the KSTE encoder and the sieve. The bit-identicality property is maintained because the standard decoder doesn't consult the KSTE structure; it reads the Spinor block, which is unchanged.

4.2 ARM aggregation (`SP_LATTICE_ARM`)

ARM (Algebraic Resonance Memory) is an HRR-style associative memory living in the cyclotomic ring. Each bound key-value pair (k, v) contributes a single bound element in the ring;

multiple bindings accumulate via ring addition, producing a slab whose state encodes the aggregate of all bound pairs. Recall on a query key q produces a noisy approximation of the bound value, with noise scaling as 1/sqrt(K) for K bindings.

At a single node, ARM is a side-effect: bindings happen on chunk boundaries during forward inference, and recalls happen as an injection in the attention path. With the gate off, neither happens. With the gate on, ARM is *local-only* by default — bindings stay on the local node — and the contribution to forward inference is a small alpha-weighted bias on the attention scores.

Multi-node ARM is where the federated story begins. Slabs are gossiped between peers; a node receiving a slab adds it to its local bank, so over time every participating node accumulates a shared associative memory of bindings made anywhere in the network. The aggregation is the literal ring sum, which means it commutes and associates cleanly; no consensus is required to merge slabs, only delivery. This makes the ARM gossip protocol simple: each node periodically emits its slab deltas; peers integrate them on receipt.

The capacity-bound mitigation is important here. ARM bindings have a cosine-similarity capacity curve that degrades as K grows. With multi-node aggregation, K can grow without bound, so a participating node must either periodically rebind from a fresh slab or accept that older bindings become noisier. The current design is periodic rebinding: every interval (default 1 hour wall-clock), each node forgets bindings older than a threshold and starts a fresh slab. This bounds the capacity but means short-term gossip is the primary value of ARM, not long-term memory.

Golden-ratio key generation. ARM requires binding keys that are mutually near-orthogonal under circular convolution. The previous design used random projection with a Gram-Schmidt residual to enforce orthogonality; the cost was a heavy initialization pass at slab creation. The new path generates keys deterministically using φ-spaced phases in the cyclotomic ring R_q — specifically, the k-th binding key has phase $2\pi k\varphi \bmod 2\pi$, with the rest of the ring coordinates derived from a fixed seed. By Three-Gap (PPT-LAT-Theory T7), this gives near-orthogonality structurally with no Gram-Schmidt cost. The empirical capacity curve, currently $0.83 \to 0.15$ cosine recall across $K = 1, \ldots, 64$ sparse bindings, is expected to extend toward $K = 128$ or beyond under φ-spaced initialization; the validation sweep is in Phase 9 of the roadmap.

4.3 CRT-sharded inference (`SP_LATTICE_CRT_SHARD`)

CRT sharding is a way of running one inference across two nodes by splitting the NTT computation across the two coprime Proth primes q_1 and q_2. Each shard holds the weights in residues mod q_1 (one node) or q_2 (the other); the activations are sharded the same way. Forward inference proceeds in parallel on both nodes; the outputs are CRT-reconstructed at the very end to produce a single fp32 logit vector for sampling.

The bandwidth requirement between shards is small: only the residue-domain activations need to cross the wire, and those are about 30 bits per scalar (vs 32 for fp32). The latency requirement is harder, because synchronizing twice per layer (once before attention output combine, once before FFN output combine) means the inter-node round-trip lands on the per-token critical path. At 100 ms inter-node latency and a 32-layer model, this adds 6.4 seconds per token, which is a non-starter for interactive use; at 5 ms LAN latency, it adds 320 ms per token, which is workable.

The pragmatic conclusion is that CRT-sharded inference is for batch or research workloads, not for interactive chat, until we have the data center deployment we don't yet have. It is also one of the open questions: there may be a way to relax the per-layer synchronization to per-block (every 4 layers, say) at some cost in numerical accuracy, which would let CRT sharding work over WAN. We have not characterized that tradeoff.

With the gate off, CRT sharding doesn't engage; the engine runs single-node and computes the residues internally for verification. With the gate on, the engine looks for the named peer at the configured address, establishes a session, and switches to the shard protocol.

4.4 DHT participation (`SP_LATTICE_DHT`)

The DHT (Distributed Hash Table) is what makes the Lattice a network rather than a collection of isolated nodes. It uses a **2-Axis Fibonacci-Prime Address Space** — a hybrid of semantic and load routing that solves the standard Kademlia clustering problem natively without resorting to cryptographic hashing.

Axis 1 — Semantic axis (prime-factored lattice). The prime factorisation of lattice slab indices routes content based on semantic adjacency: related content lives at nearby prime indices, by construction of the KSTE encoder (PPT-LAT-Theory §4). Slabs corresponding to the same prime root are likely to hold similar content; queries against related semantic neighbourhoods naturally cluster their traffic on the same DHT subtree.

Axis 2 — Load axis (Fibonacci hashing). Pure semantic routing would concentrate traffic on popular semantic neighbourhoods, leaving other nodes idle. To prevent this, node addresses *within* a semantic slab are derived by multiplying the node ID by $\varphi = (\sqrt{5}-1)/2$ and taking the fractional part — the standard Fibonacci hashing of Knuth §6.4, which by Three-Gap (PPT-LAT-Theory T7) maximally distributes load within the slab regardless of the underlying ID distribution.

The two axes compose: the semantic axis decides *which* slab handles a request; the Fibonacci axis decides *which node within the slab* handles it. Because the two axes are mathematically independent, traffic skew on one axis does not propagate to the other — popular semantic neighbourhoods stay popular, but the nodes within them are uniformly loaded.

The DHT carries the same payload types as a single-axis Kademlia variant:

- **Sieve cache deltas.** When a node accepts a new KSTE tree on its local sieve, it gossips the tree to peers whose semantic-slab overlaps. Peers integrate the delta if it adds to their incomparable frontier.
- **ARM slab updates.** Periodically (default once per minute), each node emits its ARM slab delta. Integration is ring addition.
- **Block propagation.** New blocks (see §6) propagate via standard gossip with deduplication by block hash.
- **Sharded inference recruitment.** When a node wants a CRT-sharded inference peer, the DHT lookup runs against compatible shard assignments.

Bootstrap: `SP_LATTICE_DHT=<peer_addr>` points at a known bootstrap node; from there, standard Kademlia bootstrap (k-buckets, refresh) discovers peers along both axes.

This is also the substrate for the crawler: when the lattice sieve identifies a slab whose coverage is thin, the crawler requests dominance frontiers from peers in adjacent semantic slabs (axis 1) while the Fibonacci axis ensures the crawl load is spread across all participating nodes in the source slab.

4.5 Token-economy tracking (`SP_LATTICE_TOKENS`)

With this gate on, the node tracks two separate ledgers of credit. The first, the **work-token ledger**, accumulates as the node performs verifiable inference on behalf of others — every CRT-shard contribution, every recall against a remotely-bound ARM slab, every block-attestation it serves to peers. The work ledger increments by an amount proportional to the verified compute performed, where the unit of verified compute is a normalised matmul-op count divided by a hardware factor that prevents trivial inflation from running on a faster machine. The

second, the **discovery-token ledger**, accumulates when the node contributes a dominance-incomparable KSTE tree to the network — that is, when its local sieve identifies a cache cell that is novel against the global frontier. Discovery tokens reward bringing new information into the lattice rather than simply re-serving what is already known; they are the mechanism by which the network's coverage of the input distribution grows beyond what any single operator could supply.

Both ledgers are local until block boundaries. At the periodic block cadence (see §6), the node submits a settlement proof: a Merkle-rooted summary of the deltas accrued since the last settlement, signed by the node key and accompanied by the dominance proofs (for discovery) or verification attestations (for work) that the chain validators will check. Accepted settlements mint the corresponding tokens against the node's on-chain balance. The gate is off by default not because tracking is expensive — the bookkeeping is a few hundred bytes per layer per token — but because participation in the chain economy is opt-in: a node operator running the engine purely for local inference has no reason to incur the gossip and settlement costs. Turning the gate on is the operator's explicit consent to be a paid participant.

Section 5: Network and protocol

The Lattice network is the carrier for everything in §4 once it crosses the boundary between one node and another. This section describes how the wire works.

5.1 DHT over prime-factored lattice key space

The DHT layer is described mathematically in §4.4; here we cover the operational shape. The Lattice runs a libp2p-compatible swarm: each node holds a long-lived Ed25519 identity key, derives a peer ID from it, and joins the DHT by contacting one or more configured bootstrap peers. The transport is TCP with optional Noise encryption (default on for inter-node traffic, off for in-LAN benches where latency dominates and the messages are signed at the protocol layer anyway). QUIC is on the roadmap but not yet in production; the gain on inter-node latency is real (one round-trip saved per connection) but the maturity of the QUIC implementation on Windows hosts has not been validated.

The 2-axis routing of §4.4 is layered on top of standard Kademlia: the k-bucket table is partitioned by semantic axis (so each bucket holds peers whose semantic-slab prefix matches at the bucket's depth), and within each bucket the entries are sorted by Fibonacci-hash distance. Lookups walk the semantic axis first (closer prefix = closer logical distance) and break ties on the load axis. This composes cleanly with the standard Kademlia bootstrap and refresh logic; the only modification is the distance metric.

Peers are health-checked every 30 seconds with a small ping (16 bytes); a peer that misses three consecutive pings drops out of the routing table. Re-discovery is automatic via the next refresh cycle. The protocol is intentionally chatty rather than minimal because the underlying network conditions (residential broadband, intermittent NAT) are unreliable enough that aggressive health-checking pays for itself in fewer wedged sessions.

5.2 Wire formats

Three message types dominate the wire: KSTE trees, ARM updates, and CRT residue blobs.

KSTE trees are packed at 64 bytes per node: a 1-byte type tag, a 1-byte depth marker, a 30-bit fingerprint (split across two fp16-shaped fields for alignment), a 16-byte parent pointer (Merkle hash), and a 38-byte payload — either an internal-node delta blob or a Spinor leaf. The tree as a whole is delivered as a length-prefixed sequence of nodes in pre-order traversal, with

the root first. A typical tree is 6-10 nodes deep, so the median tree message is roughly 500 bytes. The encoding is fixed-endianness (little-endian, the dominant target architecture) and there are no embedded pointers — everything is index-relative within the message.

ARM updates are a single ring element of R_q at the configured NTT N. For our standard N=256 and dual-prime q at 60-bit reconstructed, that is 256 coefficients × 8 bytes = 2 KB per slab delta. We do not currently delta-encode (the ring sum is the delta), though there is room to: subtracting the last-acknowledged remote slab from the local slab and shipping the difference would compress well-correlated traffic. This is one of the future optimizations.

CRT residue blobs are the per-layer activations during sharded inference. They are sized by the model's hidden dimension times 30 bits per scalar (packed); for Llama 3.1 8B (d=4096) that is 4096 × 30 / 8 = 15 KB per layer per token. At 32 layers and 100 tokens/sec target generation rate, that is 48 MB/sec of inter-shard bandwidth — comfortably within a gigabit LAN's capacity but well above what residential broadband can sustain. This bandwidth shape is why CRT sharding is LAN-bound in the current generation.

5.3 Gossip and propagation

For payloads other than CRT (which is point-to-point between sharded peers), the protocol is gossip. Each node maintains a small set of *fanout peers* — typically 8 randomly chosen from the routing table, refreshed every minute — and when a new payload arrives that the node has not seen before, it forwards the payload to its fanout peers (minus the one it came from). Deduplication is by content hash; each node maintains a recently-seen set of roughly 10,000 hashes with a TTL of 5 minutes, evicting LRU.

The 2-axis routing matters here too: gossip messages are tagged with their semantic slab, and fanout peers are selected preferentially from peers whose semantic slab overlaps the message's. This biases propagation toward peers who are likely to care, without precluding rare cross-slab discovery (the 8-peer fanout is large enough to almost always include at least one out-of-slab peer per hop).

Block messages are special: they propagate aggressively (fanout of 32) and validate-on-receive (the receiving node checks the block's signatures, the validator selection, and a sample of work-token proofs before forwarding). A node that receives an invalid block does not forward it; this contains the damage from a misbehaving peer to its immediate neighborhood, where it will be observed by the slashing infrastructure described in §6.

Section 6: Blockchain design

This section is scaffolding. The chain exists to give federated participation an accounting layer; the parameters here are starting points, expected to be revised as the network grows. We have chosen to write down a specific design rather than gesture at one, because specificity is what makes the design critiquable. Where the choice is contested or unclear, we say so.

6.1 Two-token economy

There are two native tokens on the Lattice chain. The **Work token (W)** is paid out for verifiable inference contributions: serving a CRT shard, attesting blocks, responding to a remote ARM recall. The **Discovery token (D)** is paid out for contributing a dominance-incomparable KSTE tree to the global sieve — that is, for genuinely novel cache contributions that expand the network's coverage.

The two tokens are fungible against each other through an automated market maker (a Uniswap-V2-style constant-product pool), with both sides bootstrapped at genesis (see §6.5). The conversion ratio drifts with supply and demand; the design intent is that work and discovery should be roughly equally weighted at steady state, but the market is allowed to find the actual ratio.

The reason for two tokens rather than one is twofold. First, separating them lets validators tune emission policy: if the network is over-served (lots of work, few queries) then work emission can be throttled while discovery emission stays high to encourage cache expansion. Second, the two activities have different economic profiles — work is high-frequency, low-value-per-event; discovery is low-frequency, high-value-per-event — and conflating them in a single ledger would force every participant to optimize for the marginal of both, when in practice operators specialize.

A note on monetary policy: emission for both tokens follows a logarithmic schedule (block-reward shrinks slowly over time, asymptoting toward zero) rather than the abrupt halvings of Bitcoin. This is deliberate: we want predictable long-run dilution rather than the speculative volatility around halving events. The schedule is parameterised at genesis and updatable by validator vote with a six-month cooldown.

6.2 Proof-of-Useful-Work via dominance verification

The chain's core consensus primitive is not arbitrary hashing — it is *dominance verification*. To mint a Discovery token, a node must submit a KSTE tree along with a proof that the tree is incomparable against the chain's current dominance frontier (the on-chain Merkle-rooted summary of all accepted KSTE trees to date). The proof is small: a Merkle inclusion path from the chain's frontier root to the trees that bound the new tree's incomparability claim, plus the new tree itself. A validator checks the proof by recomputing the dominance comparisons against the cited bounding trees; if the new tree is genuinely incomparable, the proof verifies and the token is minted.

To mint a Work token, the node submits an *attestation* of work performed: a signed record of a CRT-shard session or an ARM-recall service, countersigned by the requester. The validator checks both signatures and ensures the work record's hash is consistent with the requester's claim. Work attestations are heavier than discovery proofs in volume but lighter per-attestation in compute; the chain budgets accordingly.

The key property is that the work being proved is *useful*: it is the same work that the user wanted done anyway. Compare to Bitcoin, where the proof-of-work is intrinsically wasteful (the SHA-256 hashing has no purpose other than rate-limiting block production). Lattice's proof-of-useful-work means that the energy spent producing tokens is the energy spent producing the inference and discovery that is the network's actual product.

The honest caveat: useful-work proof systems are harder to make adversary-resistant than wasted-work systems, because the adversary can compute on hardware that produces real outputs and is hard to distinguish from honest participation. The discovery side is partly protected by the dominance check (a forged "novel" tree must actually be incomparable to be accepted, and forging incomparability is no easier than discovering it), but the work side relies on requester countersignatures, which means a colluding requester-server pair can mint work tokens against fake sessions. The mitigation is rate-limiting and reputation, both described in §6.6.

6.3 Block structure

Blocks are produced at a target rate of one per 60 seconds. The variance of inter-block intervals under the Golden-Ratio validator rotation (§6.4) is bounded by the rotation's discrepancy

property, which is tighter than the Poisson process produced by a randomised beacon — useful because it means downstream consumers can rely on roughly-on-time blocks.

Each block contains:

- **Header.** Parent block hash, block number, timestamp, validator ID, validator signature, Merkle root of body.
- **Work attestations.** A sequence of work attestations (requester-server signed pairs) with their tokens minted accordingly.
- **Discovery commitments.** A sequence of KSTE-tree dominance proofs, each minting Discovery tokens to its submitter. The accepted trees become part of the next block's dominance frontier root.
- **ARM gossip updates.** A summary of ARM slab updates seen on the network in the last block-interval. These are not consensus-critical (ARM is best-effort gossip), but they are written into the block as a checkpoint so a fresh-joining node can reconstruct the network's current associative memory state without re-receiving the entire gossip history.
- **Sieve cache deltas.** Similarly, a summary of new KSTE trees accepted into the global sieve since the last block. These determine the next block's dominance frontier root.
- **Token mint events.** The settlement records that turn the work and discovery accruals into on-chain balance changes.
- **Validator vote slot.** Reserved space for governance votes (parameter updates, validator-set changes).

A typical block is on the order of 100 KB to 1 MB depending on network activity. Block storage grows linearly with time but the dominance frontier — the load-bearing state for new-block validation — is bounded by the natural growth rate of incomparable trees, which empirically saturates as coverage grows. We have not seen full saturation in the bench; the projection is that the frontier reaches a stable size in the low millions of trees and stops growing meaningfully thereafter.

6.4 Consensus and validator rotation

Consensus is BFT-style: a quorum of two-thirds of stake-weighted validators must sign a block for it to be canonical. The validator for any given block — the proposer — is chosen by Golden-Ratio rotation through the validator set, described next.

6.x Validator selection via Golden-Ratio rotation

Stake-weighted validator selection in standard consensus protocols requires a verifiable random beacon (VRF, randao, or similar) — a non-trivial source of consensus compute. By Three-Gap (PPT-LAT-Theory T7), we can replace the random beacon with deterministic Golden-Ratio rotation through the validator set: with stake-weighting baked into the interval lengths, stepping through the set by increments of φ guarantees mathematically optimal fairness and distribution over time without relying on pseudo-randomness.

Concretely: let V be the validator set with stake weights $w_1, \ldots, w_n$ normalised so $\sum w_i = 1$. Partition the unit interval into n arcs of length w_i. To select the validator for block b, compute $r_b = \{b \cdot \varphi\}$ (fractional part) and pick the validator whose arc contains r_b. Three-Gap guarantees this produces a uniform distribution of validator selections over time, weighted exactly by stake, with no manipulability advantage from controlling any single block.

The advantage over a random beacon is twofold: (1) no consensus rounds needed to agree on the beacon output for the next block, and (2) any node can independently verify the validator assignment for any block by computing $\{b \cdot \varphi\}$ — no commit-reveal, no slashing for beacon non-cooperation.

The validator set itself is updated by stake delegation: token holders bond W or D against a validator's address, and the validator's stake weight w_i is the sum of bonds. Bond and unbond are processed at block boundaries with a configurable unbonding delay (default 14 days) that prevents stake from being moved fast enough to manipulate near-term validator selection.

6.5 Genesis and parameterization

Genesis parameters are chosen to bootstrap a small functioning network rather than to optimize for any particular long-run equilibrium; the expectation is that everything in this section will be revised as the chain grows.

Token supply at genesis. 100,000,000 W and 100,000,000 D, distributed across a small founder set, an early-contributor set, and a treasury (held by the validator quorum for protocol grants). The founder share is restricted by linear vesting over 4 years.

Initial validator set. 21 validators, geographically distributed where possible, with stake bootstrapped from the treasury. The set expands by validator-vote as the network grows; the design ceiling is in the low hundreds of validators, above which BFT signature aggregation becomes the binding bottleneck.

NTT and CRT parameters. $N = 256$, $q_1 = 1073738753$, $q_2 = 1073732609$, reconstructed modulus $\approx 2^{60}$. These match the engine's standing configuration and the chain's residue-verification path uses the same arithmetic, so a validator can re-execute any submitted work-attestation on a single machine if it wants to.

KSTE configuration. Tree depth 8, fingerprint width 30 bits, Spinor block 63 bytes. These match the production cache encoder; the chain's dominance verification is therefore the same operation the engine already performs on every cache write.

Block parameters. 60-second target inter-block time, 2 MB block size cap, two-thirds-stake BFT quorum, 14-day unbonding delay.

These constants are encoded in the genesis block and can be modified by a validator vote with a six-month cooldown, except for the NTT/CRT primes which are fixed for the lifetime of the chain (changing them would invalidate every prior dominance proof).

6.6 Slashing

Validators and participants can lose stake for misbehavior. The chain defines four slashing conditions:

False novelty claims. A node submits a KSTE tree claiming dominance-incomparability against bounding trees that, when re-checked, do not bound the claim correctly. The submitter loses a fraction of bonded stake (default 10%) and the false claim is purged from the frontier.

Residue forgery. A CRT-shard server submits residue blobs that fail the cross-prime consistency check at the requester end. The server loses a larger fraction of stake (default 50%) and is removed from the validator set if the offense recurs.

Equivocation. A validator signs two different blocks at the same block height. The validator loses its entire stake; this is the most serious offense because it threatens consensus directly.

Censorship. A validator that is the rotation-elected proposer for a block but produces no block within the block-interval is mildly penalised (default 1% of stake) and the next-elected validator (the next Golden-Ratio arc) proposes instead. Repeated censorship leads to validator removal.

Slashing decisions are made by the validator quorum based on evidence submitted by any node; a successful slash mints a small bounty to the evidence submitter, paid from the slashed stake. This is the chain's whistleblower incentive.

6.7 Mutability — papers are scaffolding, not specification

We have emphasized throughout that this design is scaffolding. The slashing fractions, the block size cap, the unbonding delay, the emission schedule — all of these are starting points. We expect that within the first six months of mainnet operation, real adversarial pressure will reveal which constants are tight and which are slack. The validator-vote mechanism is the path for revising them.

What is *not* mutable: the NTT/CRT primes (changing them invalidates prior dominance proofs), the dominance comparison itself (it is the mathematical core), and the existence of two separate token ledgers (folding them together is a fundamental design change, not a parameter tweak). Anything else is in scope for revision.

This is what we mean by "papers are scaffolding, not specification." The mathematical core (PPT-LAT-Theory) is fixed; the systems paper is current; the chain parameters are a snapshot of the design at this date and will be revised. A future systems paper will reflect those revisions rather than pretending the constants were oracular from the start.

Section 7: Failure modes and mitigations

Every networked system has failure modes; a useful systems paper names them rather than pretending they don't exist.

Sybil attacks. An adversary spawns many cheap identities to dominate the DHT and the validator set. Mitigation: stake-weighting on the validator side means Sybil identities have no consensus weight unless they buy stake on open market; on the DHT side, the 2-axis routing of §4.4 limits the damage a single semantic-slab cluster can do, and the Fibonacci-load axis spreads any honest traffic across the cluster's nodes rather than concentrating it on the adversary's. Sybils can still wedge gossip — by accepting and dropping payloads instead of forwarding — but the bound on damage is local rather than global.

Cache poisoning. An adversary submits KSTE trees designed to be incomparable but to point toward arbitrary cache content, polluting the global sieve. Mitigation: dominance verification at write time means the cell content matters, not just the tree shape; a poisoned cell still has to be incomparable on its actual content, which limits the adversary to genuinely novel-by-content submissions. This doesn't prevent semantic poisoning (a cell that is genuine-but-misleading), but it does prevent the most trivial attacks.

Eclipse attacks. An adversary surrounds a target node with adversarial peers, controlling its view of the network. Mitigation: the 2-axis routing makes eclipse harder than in single-axis Kademlia, because the target's routing table is partitioned by semantic slab and the adversary must control sufficient peers in each slab to be effective. Additionally, the bootstrap-peer flag accepts a comma-separated list of peers, and the target node prefers diverse bootstrap peers (different ASNs, different geographic origins where available) for the initial routing-table seed.

Free riders. Operators run nodes that consume bandwidth and serve queries but never propagate gossip or contribute KSTE trees back to the network. Mitigation: this is a recurring problem in P2P systems and the only known robust solution is incentive alignment, which is what the two-token economy provides. A node that does not gossip earns no work tokens; a node that does not discover earns no discovery tokens. Pure consumers are tolerated (the network can

absorb them at the margin) but they cannot scale, because they have no on-chain identity to bond stake against.

ARM capacity overflow. As more bindings are gossiped into a node's ARM bank, recall noise grows. Mitigation: periodic rebinding (§4.2) bounds the working set; nodes that wish to retain long-term associative memory must do so out-of-band (a separate persistence layer outside the gossiped slab). The chain itself does not store ARM state, only summary checkpoints in blocks.

Cache staleness. A KSTE tree that was incomparable when submitted may become dominated as the frontier grows. The chain treats this as natural state change — the original Discovery token is not clawed back, but the tree's contribution to the current frontier is just whatever it currently is. This is the right behavior because it preserves the property that Discovery rewards genuine novelty at the time of contribution.

Network partitions. A split in the gossip layer (caused by a major ISP outage or backbone failure) causes the network to operate as two disjoint sub-networks for some interval. Mitigation: the chain uses a BFT quorum, so a sub-network with less than 2/3 stake cannot produce blocks. When the partition heals, the longer-chain rule applies (the sub-network that did produce blocks wins). Sub-network participants who minted tokens during the partition are not penalized for the partition itself, but their gossip-deltas are only applied to the canonical chain after the partition heals.

Collusion. A coalition of validators with combined stake greater than 1/3 can stop block production by refusing to sign; a coalition with combined stake greater than 2/3 can produce arbitrary blocks. Mitigation: this is the standard BFT bound and we don't claim to do better than it. The chain's defense is the social one of decentralized stake distribution; the chain protocol provides no cryptographic defense against a stake-supermajority cartel. The treasury holdings at genesis are deliberately distributed to make supermajority capture expensive.

Section 8: Comparison to prior work

The Lattice sits in a crowded design space. This section names the closest neighbors and where the Lattice diverges.

Hivemind / Petals. These projects pioneered the idea of running large LLMs across volunteer GPUs over the public internet. Petals in particular ships working multi-node inference across heterogeneous home GPUs for BLOOM-176B and Llama-class models. The Lattice differs principally in (a) the cache and weight compression that lets it run on phone-class hardware, not just GPU peers, and (b) the on-chain accounting layer that allows participants to be paid for contributions, which Petals deliberately avoided. Petals is a closer-to-academia project; the Lattice is a closer-to-production one. We have used Petals as a reference for the gossip and routing layer design, but the wire formats and consensus model are not shared.

Bittensor. Bittensor implements a peer-to-peer market for ML model outputs with on-chain incentives. The closest analogue to our work-token. Bittensor's contribution is the idea that contribution can be measured by peer-ranking against a shared benchmark; the Lattice's discovery-token has the same flavor but measures contribution by dominance frontier expansion rather than by peer ranking. The two systems could coexist — a Bittensor subnet running on top of a Lattice cache substrate is plausible — but they are not the same system.

DiLoCo. Federated training rather than inference, but the geographically-distributed compute story is similar. DiLoCo's contribution is showing that the gradient-sync frequency can be relaxed without hurting convergence, which lets training tolerate WAN latencies. The Lattice's CRT-sharded inference faces an analogous latency-tolerance question, and we are watching

DiLoCo's results to inform our roadmap on relaxed-synchronization sharding. DiLoCo is research; the Lattice is a production target with a research roadmap.

Filecoin / Storj. Storage networks rather than compute, but the on-chain accounting layer for verifiable-useful-work is the closest precedent for what we are doing with proof-of-useful-work. Filecoin's proof-of-replication and proof-of-spacetime are different cryptographic primitives but the same systemic role — they are the chain's way of verifying that the network's product (stored data, in Filecoin's case) was actually produced. We owe Filecoin a conceptual debt for showing that this design pattern is viable at production scale.

YaCy. A peer-to-peer search engine, dormant but instructive. YaCy showed that a distributed crawl and index can be operated by a volunteer network; it failed to reach critical mass partly because there was no incentive layer to reward crawlers. The Lattice's discovery-token is in part a response to that lesson: making the cache contribution itself the rewarded act, rather than relying on volunteer altruism.

Render Network. A blockchain-coordinated marketplace for GPU rendering work. Closer to a generic compute marketplace than to Lattice's tightly-coupled inference fabric, but it is the closest production example of compute-for-tokens at scale. Render's accounting layer is simpler than ours (no useful-work proof; the work is just verifiably-rendered frames) and Render's compute jobs are embarrassingly parallel in ways inference is not. The mechanisms are different but the political economy is similar.

Bitcoin / Ethereum. The base layer. Bitcoin's contribution is the validator-rotation primitive; Ethereum's contribution is the smart-contract substrate that lets economic logic be expressed declaratively. The Lattice borrows the BFT-quorum consensus model from Ethereum's post-merge design, the on-chain settlement-record pattern from both, and the validator-slashing playbook from both. The Lattice does not currently have a general-purpose smart-contract layer; the chain logic is fixed at protocol level. Adding a general VM is on the longer-term roadmap but is not a near-term priority because the use case is small (governance votes and treasury management, mainly).

Section 9: Open questions

This section names what we don't know. We name them rather than gesture at them because being precise about what is unknown makes the work bounded.

Constants under real churn. The slashing fractions, unbonding delay, fanout sizes, and gossip refresh intervals are all educated guesses at this point. We do not have data from a churning production network to validate any of them. We will know more after six months of mainnet operation; until then, these constants are placeholders.

Two-token equilibrium proof. We have argued informally that a two-token system with separate work and discovery rewards should not collapse to a single token via the AMM, because the two activities have different supply dynamics. We do not have a formal equilibrium proof, and it is possible that under some demand regime the discovery side dries up (if all easy novel content has been discovered) or the work side does (if there is no demand for inference). The chain has parameter levers to respond to this, but we have not characterized when those levers need to be pulled.

Slab assignment under adversarial churn. The 2-axis DHT assumes that semantic slabs are stable enough that a node's slab assignment is a meaningful long-term identity. If churn is very high — nodes leaving and rejoining with different IDs frequently — the slab assignment becomes noisier and the routing benefit degrades. We do not know what level of churn breaks this; the literature on Kademlia variants under churn is helpful but doesn't directly address the 2-axis case.

Real-time vs batch tradeoff for CRT sharding. §4.3 frames CRT sharding as LAN-bound for interactive use. Real-time WAN sharding would require relaxing per-layer synchronization, which is an open numerical-accuracy question. We have not characterized how much synchronization can be relaxed before perplexity drift becomes unacceptable.

Validator hardware diversity. Block validation requires re-checking dominance proofs and re-executing CRT residue checks. If validators all run the same hardware, a hardware bug affects every validator simultaneously, which is a consensus risk. We want validators to run a diversity of hardware (Intel/AMD/ARM, different vendors of compute accelerators) to provide implementation diversity at the consensus layer. We do not know how to incentivize this beyond exhortation; making it economically rational is an open design question.

Privacy of shared KV cache. A shared cache substrate inherently leaks information about what was queried: an adversary monitoring the gossip layer learns the semantic distribution of queries on the network. For consumer use cases this is acceptable (the queries are not private), but for enterprise or regulated workloads it is not. We do not currently have a privacy-preserving variant; the most plausible route is to layer secure multi-party computation on top of the CRT-shard protocol, but the bandwidth and latency cost of MPC is likely prohibitive for interactive inference. This may be the open question that defines the next generation of the design.

Section 10: Anti-contamination

This project rebuilds from scratch. The Lattice engine, network, and chain described in this paper are new code, not lifted from prior repos. The conceptual debt to the PPT (Prime Position Theory) family of math papers is acknowledged — the lattice object, the CRT decomposition, the dominance partial order, the Three-Gap construction, the ARM math, the KSTE encoder, are all developed in PPT-LAT-Theory and its companions. The systems paper here describes how those mathematical objects are realized in production code; no source code is carried over from prior engine implementations into the shannon-prime-lattice repository.

Specifically, no reads occur during this paper's drafting from `D:\F\shannon-prime-repos\shannon-prime\` or `D:\F\shannon-prime-repos\shannon-prime-engine\`. The conceptual reference is the math papers (PPT-LAT-Theory and the four-paper PPT series) plus the design discussions captured in companion documents in this repo. Where this paper cites concrete numbers (PPL gaps, dispatch ceilings, memory footprints), those numbers are reproduced from the math-paper benchmarks or from this project's own bench logs, not lifted from any other code base.

The clean-room rebuild is intentional. Earlier engine code accumulated invariants and assumptions that are hard to disentangle from the math; rebuilding lets us pick what we keep, write it down in this paper, and ship it as a coherent design rather than as the accumulated state of a longer-running project.